

JWT Authentication for Airbnb Frontend — Professional Guide

JWT Authentication for Airbnb Frontend — Professional Guide

1. WHAT JWT REALLY IS

JWT means JSON Web Token.

A secure string that contains user information.

Example payload:

```
{  
  "token_type": "access",  
  "exp": 1778356394,  
  "iat": 1778352795,  
  "jti": "f008ae58453048c5b1d8382cd92f14df",  
  "user_id": "41",  
  "username": "tolu",  
  "email": "tolu@gmail.com",  
  "role": "host",  
  "is_host_approved": true,  
  "user_message": ""  
}
```

2. HOW LOGIN WORKS

Frontend sends email and password to backend.

Backend verifies credentials.

Backend returns:

- Access Token
- Refresh Token

3. ACCESS TOKEN VS REFRESH TOKEN

Access Token:

- Short lifespan
- Used for authenticated requests

Refresh Token:

- Long lifespan
- Used to generate new access tokens

4. REAL INDUSTRY FLOW

User logs in

Backend returns access + refresh token

Frontend stores them

Frontend uses access token

Access token expires

Frontend uses refresh token

Backend returns new access token

5. UNDERSTANDING YOUR TOKEN

"user_id": logged in user

"role": determines permissions

"is_host_approved": controls host features

6. JWT STRUCTURE

HEADER.PAYLOAD.SIGNATURE

Header:

Contains algorithm and token type

Payload:

Contains user information

Signature:

Used to verify token authenticity

7. FRONTEND AUTH ARCHITECTURE

src/

■ ■ ■ context/

■ ■ ■ ■ AuthContext.jsx

■ ■ ■ services/

■ ■ ■ ■ api.js

■ ■ ■ utils/

■ ■ ■ ■ auth.js

■ ■ ■ pages/

■ ■ ■ components/

8. INSTALL LIBRARIES

npm install jwt-decode axios

9. DECODING TOKENS

```
import { jwtDecode } from "jwt-decode";

const token = localStorage.getItem("access");
const decoded = jwtDecode(token);
```

10. SAVING TOKENS

localStorage.setItem("access", access)

localStorage.setItem("refresh", refresh)

11. AUTH CONTEXT

```
import { createContext, useEffect, useState } from "react";
import { jwtDecode } from "jwt-decode";
```

```
export const AuthContext = createContext();
```

12. DYNAMIC NAVBAR

```
const { user, logout } = useContext(AuthContext);
```

13. TOKEN EXPIRATION

```
const currentTime = Date.now() / 1000;

if (decoded.exp < currentTime) {
  console.log("Token expired");
}
```

14. PROFESSIONAL AXIOS SETUP

```
import axios from "axios";

const API = axios.create({
  baseURL: "http://127.0.0.1:8000"
});
```

15. AUTO REFRESH TOKENS

When backend returns 401:

- Frontend sends refresh token
- Backend returns new access token
- Original request retries automatically

16. PROTECTED ROUTES

```
import { Navigate } from "react-router-dom";
```

17. INDUSTRY BEST PRACTICES

DO:

- Use short-lived access tokens
- Use refresh tokens
- Auto refresh tokens
- Protect routes
- Centralize auth state

DON'T:

- Store passwords
- Trust frontend authorization
- Expose admin logic only on frontend

18. FINAL MINDSET

You are building real authentication architecture used in modern production systems.